

Skill - Week10

Session 19:

1. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events

```
GNU nano 8.7.1 signal_handler.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t interrupt_count = 0;
volatile sig_atomic_t terminate_requested = 0;
volatile sig_atomic_t user_signal_count = 0;

/* Handle Ctrl+C */
void handle_sigint(int sig)
{
    (void)sig;
    interrupt_count++;
}

/* Handle termination signal */
void handle_sigterm(int sig)
{
    (void)sig;
    terminate_requested = 1;
}

/* Handle user-defined signal */
void handle_sigusr1(int sig, siginfo_t *info, void *context)
{
    (void)sig;
    (void)context;

    user_signal_count++;

    if (info != NULL)
        user_signal_count += 0;
}
```

```
root@DESKTOP-QU0HVJI:~/SKILL/skill-10# nano signal_handler.c
root@DESKTOP-QU0HVJI:~/SKILL/skill-10# gcc -Wall -Wextra signal_handler.c -o signal_handler
root@DESKTOP-QU0HVJI:~/SKILL/skill-10# ./signal_handler
Signal handlers registered successfully.
Process ID: 1548

Press Ctrl+C to test SIGINT.
Send SIGUSR1 using 'kill -USR1 PID'.
Send SIGTERM to stop the program.

^CSIGINT received. Count: 1

^Z
[3]+  Stopped                  ./signal_handler
root@DESKTOP-QU0HVJI:~/SKILL/skill-10# |
```

Report: Signal handlers were registered using `sigaction()`. The program demonstrates interrupt handling, signal actions, signal context through `SA_SIGINFO`, and safe termination using signal flags.

2. To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
#include <termios.h>

#define MAX_JOBS 10

struct Job
{
    pid_t pid;
    int status;    // 0 = Running, 1 = Stopped, 2 = Completed
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

pid_t shell_pgid;
int terminal_fd = STDIN_FILENO;

void add_job(pid_t pid)
{
    if (job_count < MAX_JOBS)
    {
        jobs[job_count].pid = pid;
        jobs[job_count].status = 0;
        job_count++;
    }
}

void update_jobs()
{
    int status;
    pid_t pid;
```

```
root@DESKTOP-QU0HVJI:~/SKILL/skill-10# gcc -Wall -Wextra -pthread interrupt_shell.c -o interrupt_shell
root@DESKTOP-QU0HVJI:~/SKILL/skill-10# ./interrupt_shell
Shell PID: 1633
Shell process group: 1633

--- MINI SHELL ---
1. Run Foreground Job
2. Run Background Job
3. Show Job Table
4. Resume Stopped Job
5. Exit
Enter choice: 1
Foreground job started. Press Ctrl+C to interrupt.
Child PID 1634 started.
Child PID 1634 running: 1
Child PID 1634 running: 2
Child PID 1634 running: 3
Child PID 1634 running: 4
Child PID 1634 running: 5
Child PID 1634 running: 6
^C
[5]+  Stopped                  ./interrupt_shell
root@DESKTOP-QU0HVJI:~/SKILL/skill-10#
```

Report: The program demonstrates signal handling and job control by protecting the shell from SIGINT, transferring terminal control to foreground jobs, and tracking stopped, running, and completed processes.

Session 20:

1. To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow

```
GNU nano 8.7.1                                suspend_jobs.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_JOBS 10

struct Job
{
    pid_t pid;
    int status; // 0 = Running, 1 = Stopped, 2 = Completed
    int progress;
};

struct Job jobs[MAX_JOBS];
int job_count = 0;
pid_t shell_pgid;
int terminal_fd = STDIN_FILENO;
/* Add a process to the job table */
void add_job(pid_t pid)
{
    if (job_count < MAX_JOBS)
    {
        jobs[job_count].pid = pid;
        jobs[job_count].status = 0;
        jobs[job_count].progress = 0;
        job_count++;
    }
}

/* Update job states using waitpid */
void update_jobs()
{
    int status;
    pid_t result;
    for (int i = 0; i < job_count; i++)
```

```
root@DESKTOP-QU0HVJI:~$ nano suspend_jobs.c
root@DESKTOP-QU0HVJI:~$ gcc -Wall -Wextra suspend_jobs.c -o suspend_jobs
root@DESKTOP-QU0HVJI:~$ ./suspend_jobs
Shell PID: 1528
```

```
--- JOB CONTROL SHELL ---
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Exit
Enter choice: 1
Foreground job started. Press Ctrl+Z to suspend.
Child PID 1529: Progress = 0
Child PID 1529: Progress = 1
Child PID 1529: Progress = 2
Child PID 1529: Progress = 3
^Z
SIGTSTP received. Job suspended.
```

```
--- JOB CONTROL SHELL ---
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Exit
Enter choice: 2

----- JOB TABLE -----
Job 1 | PID: 1529 | Progress: 0 | Status: Stopped
```

```
--- JOB CONTROL SHELL ---
1. Create Foreground Job
2. Show Job Table
3. Resume Stopped Job
4. Exit
Enter choice: 3

----- JOB TABLE -----
Job 1 | PID: 1529 | Progress: 0 | Status: Stopped
Enter job number to resume: 1
Job resumed. Press Ctrl+Z to suspend again.
Child PID 1529: Progress = 4
Child PID 1529: Progress = 5
Child PID 1529: Progress = 6
Child PID 1529: Progress = 7
Child PID 1529: Progress = 8
Child PID 1529: Progress = 9
^Z
Job suspended again.
```

Report: The program captures terminal suspension through SIGTSTP, detects stopped processes using waitpid() and WUNTRACED, maintains job states, and resumes processes using SIGCONT while preserving their execution progress.

2. To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```
GNU nano 8.7.1                                process_groups.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_JOBS 10

struct Job
{
    pid_t pid;
    pid_t pgid;
    int status; // 0 = Running, 1 = Stopped, 2 = Completed
};
struct Job jobs[MAX_JOBS];
int job_count = 0;
pid_t shell_pgid;
int terminal_fd = STDIN_FILENO;
/* Add a job to the job table */
void add_job(pid_t pid, pid_t pgid)
{
    if (job_count < MAX_JOBS)
    {
        jobs[job_count].pid = pid;
        jobs[job_count].pgid = pgid;
        jobs[job_count].status = 0;
        job_count++;
    }
}
/* Display jobs and their process groups */
void show_jobs()
{
    printf("\n----- JOB TABLE ----- \n");
    for (int i = 0; i < job_count; i++)
    {

```

```
root@DESKTOP-QU0HVJI:~$ nano process_groups.c
root@DESKTOP-QU0HVJI:~$ gcc -Wall -Wextra process_groups.c -o process_groups
root@DESKTOP-QU0HVJI:~$ ./process_groups
Shell PID: 1626
Shell PGID: 1626

--- PROCESS GROUP MENU ---
1. Create Foreground Process Group
2. Show Job Table
3. Resume Stopped Job
4. Send SIGTERM to Job Group
5. Exit
Enter choice: 1

Created child PID: 1627
Assigned process group ID: 1627
Terminal transferred to foreground job.
Press Ctrl+C to terminate or Ctrl+Z to suspend.

Child PID: 1627
Child PGID: 1627
Child process running: 1
Child process running: 2
Child process running: 3
Child process running: 4
^Z
Child process stopped.
Terminal control restored to shell.
```

```
--- PROCESS GROUP MENU ---
1. Create Foreground Process Group
2. Show Job Table
3. Resume Stopped Job
4. Send SIGTERM to Job Group
5. Exit
Enter choice: 3

----- JOB TABLE -----
Job 1 | PID: 1627 | PGID: 1627 | Stopped
Enter job number to resume: 1
SIGCONT sent to process group 1627.
Child process running: 5
Child process running: 6
Child process running: 7
Child process running: 8
Child process running: 9
Child process running: 10
^ZJob suspended again.
Terminal restored to shell.
```

Report: Process groups were created using `setpgid()`, terminal control was transferred and restored using `tcsetpgrp()`, and group signals were coordinated using `kill()` with negative PGIDs. Process states were verified using `waitpid()`.